

Transaction Isolation Levels

id: sd-001 | Sub-topic: databases | Confidence: *(pending)*/5 First logged: 2026-08-19 | Last updated: 2026-08-25 | Sessions: 1

Source Material (what I brought in)

From the system-design PDF. User's summary:

"Database isolation levels (which are used to allow a transaction to execute as if there are no other concurrently running transactions). There are 4 levels ([lowest] read uncommitted, read committed, repeatable read, serializable [highest]). This is done using Shared Locks and Exclusive Locks and something called MVCC (Multi Version Consistency Control). There was a small example using repeatable read to explain how MVCC works but it was ambiguous and not really helpful."

Requested: full detail on the topic and every term in it, assuming no background beyond basic CS.

Correction Pass

Three things to fix, one of them load-bearing:

1. **"allow a transaction to execute as if there are no other concurrently running transactions"** — that is the definition of **SERIALIZABLE** only. It is not what isolation levels *are*; it is what the strongest one gives you. Isolation levels exist precisely to let you **buy less than that** in exchange for throughput. Defining the category by its top member hides the entire point of the category.
2. **"MVCC (Multi Version Consistency Control)"** — Multi-Version **Concurrency** Control. Not a nitpick: it is a concurrency-control mechanism, sitting in the same slot as locking. It is not a consistency model, and it does not by itself decide an isolation level.
3. **"This is done using Shared Locks and Exclusive Locks and MVCC"** — these are two *competing* implementation families, not one combined mechanism. The SQL standard mandates **neither**. It defines the levels purely by which anomalies they forbid, and any engine may hit those targets however it likes. This is why "repeatable read" means materially different things in MySQL and PostgreSQL.

Correct as stated: the four names and their ordering, and that all four are attempts at the same underlying problem.

What It Is

An isolation level is a **contract about which concurrency anomalies the database is allowed to expose to you**, chosen per transaction, trading correctness guarantees for throughput.

It is a dial, not a feature. Turning it up costs latency and aborts; turning it down costs you invariants you may not realize you depend on.

The Problem It Solves

Foundations, from the bottom

A **transaction** is a group of reads and writes the database treats as a single unit. The classic guarantee bundle is ACID: **A**tomicity (all-or-nothing), **C**onsistency (constraints hold), **I**solation (this topic), **D**urability (survives a crash).

Notation used throughout, from Berenson et al. (1995) — it's compact and worth learning:

```
r1[x]    transaction 1 reads item x
w1[x]    transaction 1 writes item x
c1       transaction 1 commits
a1       transaction 1 aborts
r1[P]    transaction 1 reads the set of rows matching predicate P
```

Why not just run them one at a time?

Serial execution is trivially correct. It's also unusably slow: a transaction spends most of its life waiting on disk and network, and a serial database would leave the CPU idle through all of it while every other client queues.

So databases **interleave** operations from concurrent transactions. That is the source of every problem below.

The correctness bar

Serializability: the outcome of running transactions concurrently must equal the outcome of running them one at a time in *some* serial order.

Note "some". Not a specific order — you don't get to pick, and the database doesn't promise the order matches wall-clock start times. Any serial order counts as correct.

The anomalies (what you're buying protection from)

These are the actual vocabulary. Levels are defined by which of these they forbid.

#	Name	History	What it means
P0	Dirty Write	<code>w1[x]...w2[x]...(c1 or a1)</code>	T2 overwrites data T1 wrote but hasn't committed. Rolling back T1 now can't restore a sane state.
P1	Dirty Read	<code>w1[x]...r2[x]...(c1 or a1)</code>	T2 reads T1's uncommitted write. If T1 aborts, T2 acted on data that never existed.
P2	Non-repeatable / Fuzzy Read	<code>r1[x]...w2[x]...c2...r1[x]</code>	T1 reads x twice, gets different values, because T2 committed in between.
P3	Phantom	<code>r1[P]...w2[y in P]...(c1 or a1)</code>	T1 re-runs a <i>query</i> , gets a different <i>set of rows</i> — someone inserted/deleted a row matching the predicate.
P4	Lost Update	<code>r1[x]...w2[x]...w1[x]...c1</code>	Classic read-modify-write clobber. T1's update silently erases T2's.
A5A	Read Skew	<code>r1[x]...w2[x]...w2[y]...c2...r1[y]</code>	T1 reads x, then y, and gets a mutually inconsistent <i>pair</i> — each valid at a different instant.
A5B	Write Skew	<code>r1[x]...r2[y]...w1[y]...w2[x]...(c1 and c2)</code>	Two transactions read an overlapping set, write <i>disjoint</i> rows, and jointly break a constraint neither broke alone.

P2 vs P3 is the distinction people get wrong. P2 is a *row's value* changing under you. P3 is *set membership* changing under you — a row appearing or vanishing from a query result. You can hold a lock on every row you read (killing P2) and still get P3, because you cannot lock a row that does not exist yet. That single fact is why “repeatable read” and “serializable” are separate levels at all.

The standard's table

Level	Dirty read	Non-repeatable read	Phantom	Serialization anomaly
Read uncommitted	Possible	Possible	Possible	Possible
Read committed	Not possible	Possible	Possible	Possible
Repeatable read	Not possible	Not possible	Possible	Possible
Serializable	Not possible	Not possible	Not possible	Not possible

Read this as a **permission** table, not a behavior table. It says what an engine *may* expose, not what it *will*. An engine is free to be stricter than the level requires — and in practice they all are, in different places, which is the source of most portability bugs.

What “Serialization anomaly” actually means, and why the table only has four columns when seven anomalies were just listed above (this was asked directly — worth stating precisely): it isn't a fifth specific pattern alongside the other three. It's the ANSI/PostgreSQL definition of non-serializability itself, used as a catch-all — PostgreSQL's own docs phrase it as *“the result of successfully committing a group of transactions is inconsistent with all possible orderings of running those transactions one at a time.”* Where “Dirty Read”, “Non-repeatable Read”, and “Phantom” each name one specific interleaving shape, “Serialization anomaly” means: whatever went wrong, the final result doesn't match any serial execution — the umbrella the named ones are specific instances of, plus everything else unnamed.

The table has exactly four columns because that's what ANSI SQL-92 literally named:

Standard column	Maps to
Dirty read	P1
Non-repeatable read	P2
Phantom	P3
Serialization anomaly	catch-all

P0, P4, A5A, and A5B are each missing for a different reason:

- **P0 (dirty write)** — every isolation level is assumed to forbid it implicitly; a column for it would read “not possible” all the way down. Berenson et al. call this a real flaw in the standard — it’s silently assumed rather than tested.
- **P4 (lost update)** — belongs to cursor-stability semantics, a level ANSI never formally defined. Not in the standard’s phenomena set at all.
- **A5A / A5B (read skew, write skew)** — not ANSI phenomena at all. Berenson, Bernstein, Gray, Melton, and the O’Neils *invented* them in the 1995 critique specifically because the four-column table can’t tell real isolation levels apart — Repeatable Read and Snapshot Isolation get identical rows in it, yet permit genuinely different bad outcomes. A5A/A5B are the vocabulary needed to distinguish them.

Concretely, write skew never gets its own column — it hides inside the Repeatable Read row’s last cell:

Repeatable read		Not possible		Not possible		Possible		Possible
						^^^^^^		
						this is where A5B lives		

The standard doesn’t name write skew, doesn’t explain it — it just leaves the door open with that one vague “Possible”. The two tables aren’t in conflict: the anomalies table is the *specific* vocabulary, the standard’s table is the *ANSI-official* one, and it is strictly less expressive. Serializable’s row is the only place the gap stops mattering, because “not possible” across all four columns already implies A5A/A5B/P0/P4 are gone too.

How It Works

Family A: Locking (pessimistic)

Shared lock (S) = read lock. Many transactions may hold S on the same item at once. Blocks writers.
Exclusive lock (X) = write lock. Exactly one holder. Blocks everyone.

	S held	X held
want S	✅ compatible	❌ block
want X	❌ block	❌ block

Mechanically, the engine keeps an in-memory **lock table** — a map from `resource → (who holds it, in what mode, who's waiting)`. Acquiring a lock means adding yourself to that map; if your requested mode conflicts with an existing holder you don’t get an error, you’re **blocked**: suspended until the entry frees up.

Two-Phase Locking (2PL): a transaction has a *growing phase* (only acquires locks) followed by a *shrinking phase* (only releases) — once you release anything, you can never acquire again. This alone guarantees serializability, but plain 2PL still allows a subtle failure: if T1 releases an X lock early (shrinking phase begins before commit), T2 can acquire an S lock, read the uncommitted value, and commit based on it. If T1 then aborts, T2’s committed result is built on a value that never

happened — a **cascading abort**. **Strict 2PL** closes the hole with one rule: all exclusive locks are held until commit or abort, full stop, no early release. That’s what every real lock-based engine implements; plain 2PL is a textbook stepping stone, not something you’ll find in production.

Here’s the part that makes the whole taxonomy click. **The four levels are just “how long do you hold read locks, and do you lock things that don’t exist yet”:**

Level	Write (X) locks	Read (S) locks
Read uncommitted	Held to commit	Not taken at all
Read committed	Held to commit	Taken, released immediately after the read
Repeatable read	Held to commit	Held until commit
Serializable	Held to commit, plus predicate/gap locks	Held until commit, plus locks on predicates/ranges

Walk it downward and every anomaly falls out mechanically:

- No S locks on read → you can read uncommitted data → **dirty read**.
- S released immediately → the row can change before you read it again → **non-repeatable read**.
- S held to commit → the rows you read are frozen. But a *new* row matching your query can still be inserted, because you locked rows, and you cannot lock the absence of a row → **phantom**.
- Lock the *predicate* (or the index range / the gaps between keys) → the insert itself is blocked → phantoms gone → **serializable**.

Worked timelines — one row, `accounts.id=1` , `balance=100`

Read Committed: lock held only for the statement

Time	T1	T2	Lock table
t0	BEGIN		{}
t1	SELECT balance FROM accounts WHERE id=1; -> acquire S(row1), read 100 -> statement ends, S(row1) released immediately		{}
t2		BEGIN	
t3		UPDATE accounts SET balance=90 WHERE id=1; -> acquire X(row1)	{row1: X(T2)}
t4		COMMIT -> X(row1) released	{}
t5	SELECT balance FROM accounts WHERE id=1; -> acquire S(row1), read 90		
t6	COMMIT		

T2 was never blocked — T1’s read lock was gone before T2 even started. This is the mechanism behind P2: T1’s two reads (t1, t5) returned different values because nothing held the row still between them.

Repeatable Read: lock held to commit, so blocking actually happens

Time	T1	T2	Lock table
t0	BEGIN		{}
t1	SELECT balance FROM accounts WHERE id=1; -> acquire S(row1), read 100 -> S(row1) is KEPT (repeatable read)		{row1: S(T1)}
t2		BEGIN	
t3		UPDATE accounts SET balance=90 WHERE id=1; -> request X(row1) -> X conflicts with S(T1) -> T2 BLOCKS, suspended	{row1: S(T1), queue: [T2 wants X]}
t4	SELECT balance FROM accounts WHERE id=1; -> reuses S(row1), reads 100 again (repeatable!)		
t5	COMMIT -> S(row1) released		{}
t6		-> T2's X(row1) now granted -> UPDATE proceeds, balance=90 COMMIT -> X(row1) released	{row1: X(T2)}

T1's second read at t4 is now guaranteed identical to its first — a lock physically preventing the row from changing underneath you. The price: T2 sat frozen from t3 to t5, purely waiting on T1's *transaction* to finish, not just its statement. That's why repeatable-read-via-locking hurts throughput under contention: a slow reader stalls every writer touching the rows it read.

Deadlock

Time	T1	T2
t0	BEGIN	BEGIN
t1	UPDATE accounts SET balance=90 WHERE id=1; -> acquire X(row1)	
t2		UPDATE accounts SET balance=50 WHERE id=2; -> acquire X(row2)
t3	UPDATE accounts SET balance=10 WHERE id=2; -> request X(row2) -> held by T2 -> T1 BLOCKS	
t4		UPDATE accounts SET balance=200 WHERE id=1; -> request X(row1) -> held by T1 -> T2 BLOCKS

T1 waits on a lock T2 holds; T2 waits on a lock T1 holds — a cycle in the **wait-for graph**. The engine periodically checks for these cycles and force-aborts a **victim** (usually whichever did less work / is cheaper to redo), so the other can proceed. Application code has to catch deadlock errors and retry — it's a designed-for outcome, not a bug.

Gap locks — how phantoms actually get blocked Row locks can't stop phantoms, because you can't lock a row that doesn't exist. The fix is locking the *space between* index entries:

```
Index on orders.id, existing rows: id = 10, 20, 30

T1: SELECT * FROM orders WHERE id BETWEEN 15 AND 25 FOR UPDATE;
    -> no rows match, but T1 acquires a GAP LOCK on the interval (10, 20)
    and the interval (20, 30) — the empty space its query scanned

T2: INSERT INTO orders (id) VALUES (18);
    -> 18 falls inside the gap (10, 20) that T1 locked
    -> T2 BLOCKS, even though no row at id=18 existed to lock
```

The lock is on the *gap itself*, keyed by range, not on any row. MySQL / InnoDB calls this a **next-key lock** (row lock + gap lock combined) — the concrete mechanism behind “predicate lock” for Serializable, same idea generalized from an index range to an arbitrary `WHERE` predicate.

Cost of the whole family: blocking. A long read holds S locks and stalls every writer touching those rows, plus deadlocks, resolved by aborting a victim.

Family B: MVCC (multi-version)

The core move: **never overwrite a row in place**. Every write creates a new *version* of the row, tagged with the transaction that created it. Old versions stay until no one can still need them.

Terms, precisely:

- **Transaction ID (xid)**: an integer, handed out in increasing order, one per transaction.
- **Tuple version**: a full physical copy of a row as of one write. Every `UPDATE` creates a new one; the old one isn’t touched in place.
- `xmin`: the xid that *created* this version.
- `xmax`: the xid that *superseded* this version (an `UPDATE` or `DELETE`). Empty means this version is still current.
- **Snapshot**: a small record a transaction takes once (or per statement), roughly `{xmax: <next xid to be handed out>, active: <xids currently mid-transaction>}`. This snapshot-level `xmax` is a *cutoff*, not the same field as a tuple’s `xmax` — written `S.xmax` below to keep the two apart.
- **Visibility rule**: version `v` is visible under snapshot `S` when **both**: (1) `v.xmin` is committed, `v.xmin < S.xmax`, and `v.xmin ∉ S.active` — the creator had already committed, strictly before `S` was taken; **and** (2) `v.xmax` is empty, **or** whoever superseded it hasn’t committed as of `S` (`v.xmax ≥ S.xmax` or `v.xmax ∈ S.active`).

That second clause is the one worth sitting with: a tuple can be *physically* marked superseded (`xmax` set) and still be the version you see — if the transaction that set `xmax` is invisible to you.

(Real Postgres snapshots also carry the active-xid list to handle commits that land out of xid order — folded into `S.active` above.)

The payoff, and the reason MVCC won:

Readers never block writers. Writers never block readers.

A twenty-minute analytical scan reads the versions that were current when it started, while writes proceed at full speed creating newer ones. Under pure locking that scan would freeze the table.

Writers still block writers — MVCC still needs to stop two transactions from blindly clobbering each other’s write, via **first-committer-wins**: at commit, if anyone already committed a newer version of a row you wrote since your snapshot began, you abort.

Time	T1 (xid 100)	T2 (xid 101)
t0	BEGIN, snapshot {xmax:100}	BEGIN, snapshot {xmax:100}
t1	reads balance=100	
t2		reads balance=100
t3	UPDATE balance=90 -> v1.xmax=100, v2{xmin=100,bal=90} COMMIT	
t4		UPDATE balance=90 -> engine checks: is there a committed version of row1 newer than what my snapshot (xmax:100) saw? -> YES (v2, committed by xid 100, which my snapshot didn't see) -> ABORT: "could not serialize access due to concurrent update"

T1 committed first and wins; T2 is forced to abort and retry. The check is entirely per-row: *did anyone commit a newer version of a row I'm about to write, since my snapshot started?* — and that narrowness is exactly why it's blind to write skew (see below).

Here the level maps to *when you take the snapshot*, not to lock duration:

Level under MVCC	Snapshot taken
Read committed	A fresh one per statement
Repeatable read / snapshot isolation	One, at transaction start, reused for every statement

Worked example — three phases, same table, same xids

One fixed table, three phases, the same two transactions replayed under each rule.

Phase 1 — No isolation at all

Hypothetical engine: it stores **versions**, but its read logic ignores commit status entirely — a `SELECT` just returns the newest version that exists.

Starting state (identical for all three phases): table `accounts`, one row `id = 1`, one version `v0` created by `xid 10`, long since committed.

version	xmin	xmax	balance	xmin committed?
v0	10	(empty)	100	yes

Timeline

1. T1 BEGIN → `xid 20`.
2. T2 BEGIN → `xid 21`.
3. T2: `UPDATE accounts SET balance = 500 WHERE id = 1` → creates version `v1`, and stamps `v0.xmax = 21`. T2 has **not** committed.

Row versions at this exact moment:

version	xmin	xmax	balance	xmin committed?
v0	10	21	100	yes
v1	21	(empty)	500	no — T2 still open

4. T1: `SELECT balance FROM accounts WHERE id = 1` → no commit check, newest version wins → sees v1 → **reads 500**.

5. T2 hits an error → **ROLLBACK**. v1 is discarded/invalid. The real state is v0, balance = 100.

Problem: T1 already used 500 — a value that, as far as the database is concerned, never existed. This is a **dirty read**.

Visibility rule (applies to Phases 2 and 3)

A **snapshot** s is an explicit copied set of the xids that were committed at the moment it was taken.

A version v is visible to a transaction holding snapshot s when **both** hold:

1. $v.xmin \in s$ — the version's creator had already committed when s was taken.
2. $v.xmax$ is empty, or $v.xmax \notin s$ — whoever superseded this version had not committed when s was taken, so from s's point of view the supersession hasn't happened.

The only thing that changes between Phase 2 and Phase 3 is **when the snapshot is taken**.

Phase 2 — Read Committed

Rule: a **fresh snapshot is captured at the start of every individual statement**.

Timeline

1. T1 BEGIN (xid 20). T2 BEGIN (xid 21).
2. T2 runs the same `UPDATE ... SET balance = 500` → creates v1, stamps $v0.xmax = 21$. Not committed.

Row versions at this moment (same as Phase 1, step 3):

version	xmin	xmax	balance	xmin committed?
v0	10	21	100	yes
v1	21	(empty)	500	no

3. T1 runs `SELECT` — **statement 1**. Fresh snapshot taken now:

Snapshot for statement 1 → **committed set = {10}** (21 is absent: T2 hasn't committed)

version	<code>xmin ∈ {10} ?</code>	xmax	<code>xmax ∉ {10} ?</code>	visible?
v1	no — xmin is 21	—	—	no
v0	yes — xmin is 10	21	yes — $21 \notin \{10\}$	yes

→ T1 **reads 100**. The dirty read is prevented: uncommitted v1 is invisible.

4. T2 **COMMITs**. xid 21 is now committed.

5. T1 runs the identical `SELECT` again — **statement 2, same still-open transaction**. Read Committed takes a **brand new snapshot**:

Snapshot for statement 2 → committed set = `{10, 21}`

version	<code>xmin ∈ {10,21} ?</code>	xmax	<code>xmax ∉ {10,21} ?</code>	visible?
v1	yes — 21 is in the set	<i>(empty)</i>	n/a — empty passes	yes

→ T1 **reads 500**.

Problem: T1 ran the same query twice inside one still-open transaction, wrote nothing itself, and got 100 then 500 — a **non-repeatable read**. Read Committed can't stop it because it takes a new snapshot per statement, so anything that commits between two statements becomes visible to the second one.

Phase 3 — Repeatable Read

Rule: **exactly one snapshot, captured at BEGIN**, reused for every statement until the transaction ends.

Timeline

1. T1 **BEGIN** (xid 20). Snapshot captured right now:

Snapshot_T1 → committed set = `{10}` — reused for T1's entire transaction, no matter what commits later.

2. T2 **BEGIN** (xid 21).

3. T1 runs `SELECT` — **statement 1**, using `Snapshot_T1 = {10}`. Only v0 exists so far → T1 **reads 100**.

4. T2 runs `UPDATE ... SET balance = 500` (creates v1, stamps `v0.xmax = 21`) and **COMMITs**.

Row versions now:

version	xmin	xmax	balance	xmin committed?
v0	10	21	100	yes
v1	21	<i>(empty)</i>	500	yes

5. T1 runs the identical `SELECT` — **statement 2**. Repeatable Read **reuses** `Snapshot_T1 = {10}` ; it does not take a new one:

version	<code>xmin ∈ {10} ?</code>	xmax	<code>xmax ∉ {10} ?</code>	visible?
v1	no — xmin is 21	—	—	no — invisible even though 21 really did commit
v0	yes — xmin is 10	21	yes — $21 \notin \{10\} \rightarrow$ “not yet superseded”	yes

→ T1 **reads 100** again.

Result: T1’s two reads agree (100, 100) even though T2 genuinely committed a change in between. The **non-repeatable read** is prevented — nothing was blocked; the snapshot simply never changed after `BEGIN` .

Tying it together

The exact same sequence of events ran three times with the same xids; the only variable was the snapshot rule — ignore commit status (dirty read), one snapshot per statement (non-repeatable read), one snapshot per transaction (repeatable). No locks were taken anywhere in this example, and no writer ever waited on a reader. Write skew isn’t fixed by either of these levels — that needs something stronger (SSI or an explicit `FOR UPDATE` , both covered further down).

(Real Postgres snapshots also carry an active-xid list to handle out-of-order commits; the plain “committed set” model above gives the correct answer for every case here.)

What T1 sees under Repeatable Read is not “stale” in any sloppy sense — it is a genuine, complete, committed state of the database, just an earlier one. MVCC gives you *a consistent past*, not *the uncertain present*.

Why Repeatable Read matters in practice

Fair pushback that came up here: for a single isolated read, wanting the *freshest* value is correct — that’s exactly why Read Committed is the default in PostgreSQL, Oracle, and SQL Server. A single re-read of one row can’t actually demonstrate why you’d want the second read to match the first, because there’s only one thing being read. The value shows up once a transaction reads **two different things** and needs them to agree.

Reframed: Repeatable Read isn’t “prefer old data.” It’s — **when a transaction touches more than one piece of data, all of it should describe one real moment in time, not a mash-up of two moments that each existed individually but never together**. This is the read-skew (A5A) anomaly, worked through:

Two rows: `checking = 700` , `savings = 300` . Total assets = 1000. A report transaction wants to print that total.

```
T1 (report, Repeatable Read)
BEGIN — snapshot frozen here
SELECT balance FROM checking -> 700
```

```
SELECT balance FROM savings -> ?
```

```
T2 (transfer 200: checking -> savings)
```

```
BEGIN
UPDATE checking SET balance = 500
UPDATE savings SET balance = 500
COMMIT
```

Under **Read Committed**: the second `SELECT` takes a fresh snapshot, sees T2's commit, reads `savings = 500`. T1 prints `700 + 500 = 1200` — wrong. It pairs checking's *pre-transfer* value with savings' *post-transfer* value; the database was never actually in that combined state. \$200 appears to have materialized from nowhere.

Under **Repeatable Read**: the second `SELECT` reuses the original snapshot, T2's commit is invisible, reads `savings = 300`. T1 prints `700 + 300 = 1000` — correct, and it's the real state the database was actually in at the instant T1 began.

Where this shows up for real:

- **pg_dump** runs its export inside a single Repeatable-Read (or Serializable) transaction specifically for this reason — *“pg_dump reads are completely isolated from any concurrent write activity, which is obtained with the isolation level repeatable read or serializable.”* Under Read Committed, a table dumped early and one dumped later could reflect different moments, producing a backup where a foreign key points at a row the dump considers missing. ([PostgreSQL docs — SET TRANSACTION](#))
- **Financial / reconciliation reports** — the checking/savings shape, generalized: any report summing or cross-checking multiple queries needs them anchored to one instant, or totals won't reconcile.
- **Multi-page / paginated exports** — a large table exported across many `SELECT ... LIMIT ... OFFSET` calls, or a join spanning several statements, needs every page to reflect the same world, or rows get duplicated, skipped, or become referentially inconsistent.

The common shape: **one transaction, multiple reads, decisions or output that depend on those reads agreeing with each other**. If a transaction only ever reads one thing once, Read Committed is strictly better — no reason to pay for Repeatable Read.

How snapshot isolation blocks phantoms (P3), mechanically

The reason SI blocks phantoms is anticlimactic given the mechanics above: **the visibility rule doesn't care whether a version came from an `INSERT` or an `UPDATE`**. A brand-new row is just a version whose `xmin` is the inserting transaction's xid — the identical check applies. If that xid isn't in your frozen snapshot's committed set, the new row is invisible, full stop, regardless of whether it's a phantom row or a new version of a row you already knew about.

Table `tasks`, two existing rows, both `assigned_to = 'alice'`, both long committed: Task A `xmin=10`, Task B `xmin=11`.

```

T1 BEGIN (xid 20). Snapshot_T1 = {10, 11}.
T1: SELECT count(*) FROM tasks WHERE assigned_to='alice';
    -> A (xmin=10 ∈ {10,11}), B (xmin=11 ∈ {10,11}) both visible -> count = 2

T2 BEGIN (xid 21).
T2: INSERT INTO tasks (assigned_to) VALUES ('alice');
    -> creates Task C: xmin=21, xmax=empty
T2 COMMIT.

T1: SELECT count(*) FROM tasks WHERE assigned_to='alice'; -- same snapshot {10,11}
    -> C: xmin=21 ∉ {10,11} -> INVISIBLE, filtered out before the predicate
        even matters, regardless of the fact that it matches 'alice'
    -> count = 2, still

```

Task C genuinely exists and genuinely matches the predicate — it’s invisible only because it’s outside T1’s frozen snapshot. Under Read Committed the second count would take a fresh snapshot, see xid 21, and return 3 — a real phantom, which is why the standard permits it at that level.

Two related cases, same mechanism:

- **Delete-based phantom** (a row vanishing on re-read): T2 deletes B — sets `B.xmax = 21`, commits. T1 re-checks B: `xmax=21 ∉ {10, 11}`, so from T1’s snapshot “the deletion hasn’t happened yet” — B stays visible.
- **Update-into-the-predicate** (a row that starts matching): an `UPDATE` that changes some other row’s status to `'alice'` creates a new version with `xmin` = the updater’s xid — invisible to T1 for the identical reason a fresh insert would be. The row’s *old* version still has its original value, so it’s correctly excluded on content grounds, not visibility grounds.

The contrast worth keeping with the locking family above: gap locks prevent phantoms by physically **blocking** the conflicting `INSERT` — T2 would sit frozen until T1 commits. Under SI, T2’s `INSERT` is never blocked at all; it runs and commits instantly. T1 simply doesn’t see it. Phantom protection isn’t a separate feature bolted onto MVCC — it falls out for free from the same per-version visibility check used for everything else, which is also why PostgreSQL’s own Repeatable Read row exceeds what the standard requires (see “The standard’s table” above).

The trap: snapshot isolation is not serializable

Freezing one snapshot for the whole transaction kills dirty reads, non-repeatable reads, and phantoms for free. So SI looks like it must be serializable.

It isn’t. **Write skew (A5B) survives**, and this is the single most important thing on this page. First-committer-wins only checks *your write-set* against what committed since your snapshot — it never asks “did anyone write a row that overlaps the rows I *read*,” only “did anyone write a row that overlaps the rows I *wrote*.” Read-write overlap across transactions is invisible to it by construction.

The paper’s canonical history H5 — `x = 50`, `y = 50`, invariant `x + y ≥ 0`, replayed with the same bookkeeping as above:

```

T1 (xid 100), snapshot {xmax:100}: reads x=50, reads y=50. Sum=100. Plans y=-40.
T2 (xid 101), snapshot {xmax:100}: reads x=50, reads y=50. Sum=100. Plans x=-40.

T1: UPDATE y SET value=-40 -> write-set = {y}
    COMMIT. first-committer-wins check: any newer committed version of
    {y} since T1's snapshot? No -> commits clean.

T2: UPDATE x SET value=-40 -> write-set = {x}
    COMMIT. first-committer-wins check: any newer committed version of
    {x} since T2's snapshot? No -> commits clean.

Final state: x = -40, y = -40. Sum = -80. Invariant violated.

```

Both checks pass independently because each transaction only checks the rows in its own write-set — `{y}` for T1, `{x}` for T2 — and those sets don't overlap. This is the exact mechanical reason write skew slips through: not a bug in the implementation, the check being narrower than the anomaly.

There is no serial order producing this result: run T1 fully first and T2 reads `y=-40` and refuses. Each transaction was individually correct against a genuinely consistent snapshot. The *pair* is not.

Berenson et al. state the relationship formally:

Remark 9. REPEATABLE READ »« Snapshot Isolation.

»« means **incomparable** — neither is stronger. SI forbids phantoms that standard RR permits; RR forbids write skew that SI permits. This is routinely misreported as “SI is stronger than RR.” It isn't; it's sideways.

Reaching real serializability

1. **Strict 2PL + predicate locks** — correct, blocks heavily.
2. **SSI (Serializable Snapshot Isolation)** — PostgreSQL 9.1+. Keeps SI's non-blocking reads, adds tracking of read/write dependencies, and aborts a transaction when a dangerous cycle is detected. PostgreSQL's predicate locks *“do not cause any blocking and therefore can not play any part in causing a deadlock”* — they exist only to flag dependencies. Cost: **false-positive aborts**. You must have retry logic. (The exact dependency-cycle shapes SSI watches for weren't walked through mechanically this session — see Open Questions.)
3. **Actual serial execution** — single-threaded (Redis, VoltDB). Correct by construction; scales only via partitioning.

Trade-offs

Choice	Buys you	Costs you	Use when
Read uncommitted	Marginally less bookkeeping	Dirty reads — you may act on data that gets rolled back	Almost never. PostgreSQL doesn't even implement it.
Read committed	No dirty reads; short/no read locks; high concurrency	P2, P3, lost updates, write skew	Default for most OLTP. Correct when each statement is self-contained.
Repeatable read / SI	Stable snapshot for the whole transaction; consistent multi-statement reads	Write skew; under MVCC, aborts on write-write conflict	Reports, multi-statement reads that must agree, read-modify-write on a <i>single</i> row.
Serializable	Every anomaly gone; you can reason serially	Throughput; blocking (2PL) or abort rate (SSI); mandatory retry loop	Cross-row invariants that must hold — balances, booking limits, on-call coverage.

Rule of thumb: the moment your correctness depends on a constraint spanning *multiple rows* that concurrent transactions could each satisfy alone, snapshot isolation is not enough. Go serializable, or take an explicit lock (`SELECT ... FOR UPDATE`) on a row that both transactions must contend for — materializing the conflict MVCC couldn't see.

Numbers That Matter

No published latency figures are cited here — isolation-level overhead is dominated by workload contention, so vendor-neutral numbers would be misleading. The figures worth memorizing are structural, not performance:

- 4 standard levels; PostgreSQL implements 3 distinct ones (*"PostgreSQL's Read Uncommitted mode behaves like Read Committed"*).
- `SQLSTATE 40001` — serialization failure. PostgreSQL: *"It is important that an environment which uses this technique have a generalized way of handling serialization failures (which always return with an SQLSTATE value of '40001')"*. Your retry handler keys on this.
- **Read-only transactions never abort under SSI:** *"only updating transactions might need to be retried; read-only transactions will never have serialization conflicts."* Hence PostgreSQL's advice to *"declare transactions as READ ONLY when possible."*

In Real Systems

Engine	Default	Notable deviation from the standard
PostgreSQL	Read committed	No real read uncommitted. Repeatable read <i>is</i> snapshot isolation and forbids phantoms (stricter than the standard). Serializable = SSI, genuinely serializable.
MySQL / InnoDB	Repeatable read	Plain <code>SELECT</code> uses a consistent snapshot from the first read. <i>Locking</i> reads take gap / next-key locks , blocking inserts into scanned ranges — so RR blocks many phantoms too. Under read committed, “ <i>gap locking is disabled</i> ”, so “ <i>phantom row problems may occur</i> ”. Serializable is implemented by silently converting plain <code>SELECT</code> to <code>SELECT ... FOR SHARE</code> .
Oracle	Read committed	No true repeatable read. Its <code>SERIALIZABLE</code> is snapshot isolation — it does not prevent write skew, so it is not serializable despite the name.

Two portability landmines follow directly:

- MySQL and PostgreSQL both say “repeatable read” and give you different guarantees (MySQL adds gap locks on locking reads; PostgreSQL gives SI).
- Oracle’s `SERIALIZABLE` and PostgreSQL’s `SERIALIZABLE` are different isolation levels wearing the same word.

Common Misconceptions

- **“Isolation levels are implemented with locks.”** Two families exist; MVCC is the dominant one in modern engines. The standard mandates neither.
- **“Repeatable read prevents phantoms.”** Under the standard, no. Under PostgreSQL and MySQL, largely yes, by different mechanisms. The name tells you nothing — check your engine.
- **“Snapshot isolation is serializable.”** No. Write skew survives, and it’s the anomaly most likely to corrupt real business invariants.
- **“Serializable means transactions run in the order I submitted them.”** It means the result matches *some* serial order. Not necessarily yours.
- **“Higher isolation = slower, uniformly.”** Under SSI, reads don’t block at all; the cost appears as *aborts* under write contention, not latency. A low-contention workload can run serializable near-free.
- **“MVCC means readers see stale data.”** They see a consistent committed *past state*, which is a different and much stronger thing than stale.
- **“A serialization failure is a bug.”** It’s the designed signal to retry. Missing retry logic is the bug.
- **“Repeatable Read only matters for re-reading the same row twice.”** That’s the textbook demo, not the motivating case — a single re-read of one row genuinely is better off fresh (Read Committed’s own default logic). The real value shows up once a transaction reads *two different things* that must describe one consistent moment — see “Why Repeatable Read matters in practice” above.

Worked Example

Hospital on-call: at least one doctor must be on call at all times. The textbook write skew, and it fails in production under a default PostgreSQL/Oracle configuration.

Table `doctors`: Alice `on_call = true`, Bob `on_call = true`. Both feel unwell and click “go off call” simultaneously.

```
-- T1 (Alice)                                -- T2 (Bob)
BEGIN ISOLATION LEVEL REPEATABLE READ; BEGIN ISOLATION LEVEL REPEATABLE READ;

SELECT count(*) FROM doctors
  WHERE on_call = true;
-- 2, so it's safe to leave                SELECT count(*) FROM doctors
                                           WHERE on_call = true;
                                           -- 2, so it's safe to leave

UPDATE doctors SET on_call = false
  WHERE name = 'Alice';                    UPDATE doctors SET on_call = false
                                           WHERE name = 'Bob';

COMMIT; ✓                                  COMMIT; ✓
```

Both succeed. **Zero doctors on call.** No dirty read, no non-repeatable read, no phantom, no write-write conflict — the two `UPDATE`s touch *different rows*, so first-committer-wins has nothing to fire on. Each transaction verified the invariant against a genuinely consistent snapshot and was individually correct.

Three fixes, in increasing order of cost:

```
-- 1. SERIALIZABLE — PostgreSQL's SSI sees the rw-dependency cycle
--    and aborts one with SQLSTATE 40001. Requires a retry loop.
BEGIN ISOLATION LEVEL SERIALIZABLE;

-- 2. Materialize the conflict: force both onto the same rows with an
--    explicit lock, converting invisible write skew into a visible
--    write-write conflict. Works at REPEATABLE READ and even READ COMMITTED.
SELECT count(*) FROM doctors WHERE on_call = true FOR UPDATE;

-- 3. Push it into the database as a constraint the engine enforces
--    (a trigger, or a materialized counter row with a CHECK), so no
--    application path can violate it regardless of isolation level.
```

Fix 2 is the one worth internalizing: **when a constraint spans rows, give the transactions a single row to fight over**. That's the general escape hatch from write skew without paying for full serializability.

Discussion Log

Session 1 — opened 2026-08-19, closed 2026-08-25

Opening explanation covered everything in [What It Is](#) through [Worked Example](#) above. Cross-questioning that followed, in order:

Q1: “in the standards table what does the column on serialization anomaly refer to? what does this anomaly mean? also you listed a number of anomalies in the table right before it but left out many of those from your standards table. Why is that?” **A:** Folded into “The standard’s table” above (the catch-all definition, and why P0/P4/A5A/A5B don’t get their own columns).

Q2: “the section on locking and mvcc is not detailed enough and does not have enough examples... explain using a more detailed example taking 2 transactions and explaining when one transaction acquires a lock and leaves it for the other, etc.” **A:** Folded into “Worked timelines” under Family A (RC vs RR lock lifetime, deadlock, 2PL vs strict 2PL, gap locks), and into the MVCC term glossary + first-committer-wins timeline under Family B above.

Q3: “MVCC is still unclear and ambiguous, can you explain to me with an example where there is a concrete table, then make the transactions happen but also show what the snapshots and the xmin, xmax are in context here... first tell a problem that occurs without any level, then how is this problem solved using first level (read committed), then show a problem that is not solved by the first level and how it is solved by the second level... use opus model for this specific prompt here.” **A:** Delegated to an Opus-run agent (mechanics pre-derived, agent only formatted/presented) — produced the three-phase progressive walkthrough now in “Worked example — three phases” above: no isolation (dirty read) → Read Committed (fixes it, but exposes non-repeatable read) → Repeatable Read (fixes that too), same xids replayed across all three so only the snapshot rule varies.

Q4: “but what exactly is the use of this? when T1 makes 2 reads without writing anything, why do we even want that it reads the same value, in practice wouldn’t it make sense that the latest value

be read at every read statement?" **A:** Folded into "Why Repeatable Read matters in practice" above (the checking/savings read-skew example and the `pg_dump` citation).

Q5: "how does SI under MVCC avoid phantom anomaly?" **A:** Folded into "How snapshot isolation blocks phantoms" above (the `tasks / alice` example, plus delete- and update-into-predicate variants).

Open Questions

- **SSI internals** — PostgreSQL's SIREAD/predicate-lock dependency-cycle detection was only covered at "it exists and flags dependencies without blocking." The specific rw-conflict shapes (dangerous structures — a pivot transaction with one in-conflict and one out-conflict edge) that actually trigger the abort weren't walked through mechanically the way locking and plain MVCC were this session. Good next-session thread if Serializable comes up again.
- **Distributed/cross-node isolation** — this whole doc is the single-node story. 2PC, Spanner-style external consistency, and how isolation composes across a sharded or replicated system are explicitly deferred — see Connections.

Connections

- `system-design/databases/transactions-acid.md` (sd-002) — split off from this session's cross-questioning. Covers transaction types and ACID in general; its Atomicity/Durability (WAL) mechanism is the same logging machinery this doc leans on for "strict 2PL holds locks to commit" and MVCC's snapshot-visibility-at-commit behavior.
- `context/concept-bank.md` → `sysd-03` (CAP theorem) — isolation is the "C" of ACID, which is a different C from CAP's consistency; worth untangling when that topic comes up.
- Future: `system-design/consistency/` docs on consensus and distributed transactions — isolation is the single-node story, linearizability the distributed one.

Sources

- PDF: user's system-design PDF, isolation levels section
- [PostgreSQL 18 docs — Transaction Isolation](#) — level table, SI/SSI, predicate locks, 40001, read-only advice
- [PostgreSQL docs — SET TRANSACTION](#) — `pg_dump`'s use of Repeatable Read/Serializable for a consistent multi-table export
- [Berenson, Bernstein, Gray, Melton, O'Neil, O'Neil — A Critique of ANSI SQL Isolation Levels \(SIGMOD 1995\)](#) — P0-P4/A5 formal definitions, snapshot isolation, first-committer-wins, H5 write skew, Remark 9
- [MySQL 8.4 docs — InnoDB Transaction Isolation Levels](#) — RR default, gap/next-key locks, RC gap-locking disabled
- [Oracle serializable is not serializable — dbi services](#) — Oracle's SERIALIZABLE is snapshot isolation